

Hooks

Introduction

- Hooks are special functions in React that let you “hook into” React’s features (state, lifecycle, context, etc.) inside function components.
- Introduced in React 16.8 (2019).
- They work only inside function components (not classes).

Why Hooks?

- **Before hooks:**
 - React used class components for state & lifecycle methods.
 - Function components were stateless (UI-only).
- **Problems with classes:**
 - Harder to reuse logic.
 - Confusing this keyword.
 - Code got messy with lifecycle methods (like `componentDidMount`, `componentDidUpdate`).
- **Hooks solved this:**
 - Function components can have state & lifecycle.
 - Simpler, cleaner, reusable code.

Counter – Without Hook

```
import React from "react";

class Counter extends React.Component {
  constructor() {
    super();
    this.state = { count: 0 };
  }

  render() {
    return (
      <div>
        <h2>Count: {this.state.count}</h2>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Increase
        </button>
      </div>
    );
  }
}

export default Counter;
```

Counter – With Hook

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="container" style={{ textAlign: "center" }}>
      <h2>Counter App</h2>
      <p style={{ fontSize: "2em", margin: "1em 0" }}>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
export default Counter;
```

Rules of Hooks

1. Hooks can be called at the top level (not inside loops, conditions, or nested functions).

```
//  Correct  
function App() {  
  const [count, setCount] = useState(0);  
}
```

```
//  Wrong  
if (someCondition) {  
  const [count, setCount] = useState(0);  
}
```

2. Hooks can be called from function components or custom hooks.
 - Not in regular JS functions.
 - Allowed in App(), Counter(), etc.

Commonly used Hooks

1. `useState` : add state to function components.
2. `useEffect` : run side-effects (API calls, timers, subscriptions).
3. `useRef` : persist values across renders, access DOM elements.
4. `useContext` : access global data (like theme, user info).
5. `useReducer` : advanced state management (like Redux).
6. `useCallback`, `useMemo` : optimize performance.

useState

- It lets you add state variables to function components.
- A state variable is data that can change over time.
- When state changes, React re-renders the component with the new data.
- Syntax:

```
const [state, setState] = useState(initialValue);
```

 - `state` : current value of the state variable.
 - `setState` : function used to update the state.
 - `initialValue` : starting value (number, string, array, object, etc.).

Working of useState

1. Initialization

- On first render, React creates a state cell (internal memory slot).
- It stores the given initialValue.
- `const [state, setState] = useState(initialValue);`
- React links `state` with that memory cell.

2. Accessing State

- The state variable always holds the latest value stored in that memory cell.
- Each render uses the current value of state.

Working of useState

3. Updating State

- When you call `setState (newValue)` → React does two things:
 1. Updates the memory cell with the new value.
 2. Schedules a re-render of the component.

Working of useState

4. Re-Rendering

- During the re-render:
 - React calls the component function again.
 - The useState hook fetches the current state value from memory (not re-initialized).
 - JSX is recalculated with the new state.
 - DOM is updated only where changes occurred (via the Virtual DOM diffing).

Working of useState

5. State Persistence

- State values persist between re-renders.
- Unlike local variables, which reset on every function call, state is managed by React internally so it “remembers” values across renders.

Key Points – useState

- useState always returns a **pair**:
[currentValue, updaterFunction]
- Calling setState doesn't immediately change the value — it **triggers React to re-render**.
- State updates may be **batched** for performance (React might combine multiple updates before rendering).
- Component state is **isolated** — one component's state doesn't affect another's.

ToDo List App

Points to understand

1. Functional Update:

```
setToDoList(list.concat(newItem));
```

vs.

```
setToDoList((prevList) => prevList.concat(newItem));
```

- `setState(newValue)` → good if the new value doesn't depend on old state.
- `setState(prev => ...)` → use this when the new value is derived from the previous state.

Points to understand

2. Communication from Child to Parent

- **Why?**

- Child components often need to send data or trigger actions in their parent.

- **How?**

- Parent passes a function as a prop to the child.
- Child calls this function when an event occurs (e.g., form submit, button click).

- **Example**

```
// Parent
<NewItem onAddItem={addNewItemHandler} />

// Child
props.onAddItem(newItem);
```


Points to understand

3. Importance of 'key' in List

- **Key** is a special prop React uses internally to identify elements in a list.
- It helps React's diffing algorithm (reconciliation) to decide:
 - Which items stayed the same
 - Which items were added
 - Which items were removed

Points to understand

3. Importance of 'key' in List

- What if missing?
 - React falls back to index as key.
 - Can cause performance issues and state mix-ups (e.g., input fields losing values).
 - React shows a warning in console.
- Best Practice
 - Use a stable unique ID (e.g., item.id).
 - Avoid index as key for dynamic lists.

useEffect

- In React, components re-render when their state or props change.
- Sometimes, we need to run side effects after render — things like:
 - Fetching data from an API
 - Subscribing to events (e.g., resize, scroll)
 - Updating the DOM manually
 - Setting up timers
- `useEffect` is a React hook that lets you run side effects in function components.

useEffect

Syntax:

```
useEffect(() => {  
  // Side effect code here  
  
  return () => {  
    // (Optional) Cleanup code here  
  };  
}, [dependencies]);
```

useEffect

Syntax:

- Effect function `() => { ... }`
 - Runs after the component has rendered.
- Dependencies array `[ ... ]`
 - Controls when the effect runs:
 - `[]` → runs only once (on mount).
 - `[someValue]` → runs when someValue changes.
 - `(no array)` → runs after every render.
- Cleanup function `return () => { ... }`
 - Runs before the effect re-runs or before the component unmounts (useful for removing event listeners, stopping timers, etc.).

UseEffect - Example

Effect runs only once when component mounts:

```
import { useState, useEffect } from "react";

function OnceOnMount() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Effect runs ONLY once");
  }, []); // runs only once when Component mounts

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <button onClick={() => setCount(count - 1)}>Decrement Count</button>
    </div>
  );
}

export default OnceOnMount;
```

UseEffect - Example

Effect runs after EVERY render:

```
import { useState, useEffect } from "react";

function EveryRender() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Effect runs after EVERY render");
  }); // no dependencies

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <button onClick={() => setCount(count - 1)}>Decrement Count</button>
    </div>
  );
}

export default EveryRender;
```

UseEffect - Example

Effect runs ONLY when **state** changes:

```
import { useState, useEffect } from "react";

function DepChange() {
  const [count, setCount] = useState(0);
  const [otherCount, setOtherCount] = useState(0);

  useEffect(() => {
    console.log(`Effect runs ONLY when count changes: count = ${count}`);
  }, [count]); // runs only when count changes

  return (
    <div>
      <p>
        Count: {count} | Other: {otherCount}
      </p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <button onClick={() => setOtherCount(otherCount + 1)}>
        Increment Other Count
      </button>
    </div>
  );
}

export default DepChange;
```


Effect Cleanup

- When you use `useEffect`, you often set up something that lives beyond a single render, like:
 - Event listeners
 - Timers / intervals
 - Subscriptions (WebSockets, APIs)
- If you don't remove these when the **component unmounts** or **before re-running the effect**, they can cause **memory leaks** or **unexpected behavior**.
- React solves this by letting you **return a cleanup function** from `useEffect`.

How Cleanup Works?

1. Initial render → Effect runs (setup happens).
2. Before next effect run → Cleanup runs (removes old setup).
3. On unmount → Cleanup runs one final time.

UseEffect - Example

With Cleanup:

```
import { useState, useEffect } from "react";

function CleanTimer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    console.log("Starting timer...");
    const interval = setInterval(() => {
      setSeconds((s) => s + 1);
    }, 1000);

    // Cleanup
    return () => {
      console.log("Cleaning up timer...");
      clearInterval(interval);
    };
  }, []); // setup once

  return <p>Seconds passed: {seconds}</p>;
}

export default CleanTimer;
```

UseEffect - Example

MultipleEffects:

```
import React, { useState, useEffect } from "react";

function MultipleEffects() {
  const [count, setCount] = useState(0);
  const [otherCount, setOtherCount] = useState(0);

  useEffect(() => {
    console.log("Effect 1: Runs on every render");
  });

  useEffect(() => {
    console.log("Effect 2: Runs only once");
  }, []);

  useEffect(() => {
    console.log("Effect 3: Runs only when count changes");
  }, [count]);

  return (
    <div>
      <p>
        Count: {count} | Other: {otherCount}
      </p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <button onClick={() => setOtherCount(otherCount + 1)}>
        Increment Other Count
      </button>
    </div>
  );
}

export default MultipleEffects;
```

useRef

- It is a React Hook that returns a can be changed direct mutable object:

```
const myRef = useRef (initialValue);
```

- Shape of myRef:

```
{ current: initialValue }
```

Use of useRef

- Accessing DOM elements directly (like `document.querySelector` in vanilla JS).
- Storing values between renders **without causing re-renders** (like an instance variable).
- Holding mutable objects (like timers, previous values, WebSocket connections).

UseRef – Example 1

Accessing DOM elements

```
import { useRef } from "react";

function InputFocus() {
  const inputRef = useRef(null);

  function handleClick() {
    inputRef.current.focus(); // directly focus input element
  }

  return (
    <div>
      <input ref={inputRef} placeholder="Type here..." />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}

export default InputFocus;
```

UseRef – Example 2

Persisting Values Between Renders

```
import { useState, useRef, useEffect } from "react";

function Timer() {
  const [count, setCount] = useState(0);
  const intervalRef = useRef(null);

  useEffect(() => {
    intervalRef.current = setInterval(() => setCount((c) => c + 1), 1000);
    return () => clearInterval(intervalRef.current); // cleanup
  }, []);

  return <h1>Timer: {count}</h1>;
}

export default Timer;
```


UseRef – Example 3

Tracking Previous State

```
import { useState, useEffect, useRef } from "react";

function PreviousValueDemo() {
  const [value, setValue] = useState("");
  const prevValueRef = useRef("");

  useEffect(() => {
    prevValueRef.current = value; // store previous value
  }, [value]);

  return (
    <div>
      <input value={value} onChange={(e) => setValue(e.target.value)} />
      <p>Current: {value}</p>
      <p>Previous: {prevValueRef.current}</p>
    </div>
  );
}

export default PreviousValueDemo;
```

Key Points - useRef

- useRef() returns an object with property `current`.
- Does not trigger re-render when `current` changes.
- Best for:
 - Grabbing DOM elements
 - Persisting values across renders (without re-rendering)
 - Storing mutable objects (timers, sockets, etc.)
- Not a replacement for state → use `useState` if UI needs to update.

What is Context?

- React normally passes data top-down through props.
- For deeply nested components, this becomes **prop drilling** (passing props through components that don't need them).
- Context provides a way to share values across the component tree without passing props manually at every level.

Prop Drilling

```
function App() {
  const user = "Linux";

  return (
    <>
    |   <h1>{`Hello ${user}!`}</h1>
    |   <Component2 user={user} />
    | </>
    </>
  );
}

function Component2({ user }) {
  return (
    <>
    |   <h1>Component 2</h1>
    |   <Component3 user={user} />
    | </>
    </>
  );
}

function Component3({ user }) {
  return (
    <>
    |   <h1>Component 3</h1>
    |   <h2>{`Hello ${user} again!`}</h2>
    | </>
    </>
  );
}

export default App;
```

useContext

- useContext is a hook that lets you consume values from a Context created by `React.createContext()`.
- Syntax:

```
const value = useContext (MyContext);
```

Steps to Use Context

Step 1: Create Context

jsx

```
const ThemeContext = React.createContext("light"); // default value
```

Step 2: Provide Context

jsx

```
function App() {  
  return (  
    <ThemeContext.Provider value="dark">  
      <Toolbar />  
    </ThemeContext.Provider>  
  );  
}
```

Steps to Use Context

Step 3: Consume Context (using `useContext`)

jsx

```
function Toolbar() {  
  const theme = React.useContext(ThemeContext);  
  return <button className={theme}>I am styled by theme: {theme}</button>;  
}
```

Example : Avoid Prop Drilling

MyContext.jsx

```
import { createContext } from "react";

const ColorContext = createContext("blue");
export { ColorContext };
```

App.jsx

```
import { ColorContext } from "../Context/MyContext";
import Parent from "../components/ContextDemo/ComponentTree";

function App() {
  return (
    <ColorContext.Provider value="green">
      <Parent />
    </ColorContext.Provider>
  );
}
export default App;
```


Example : Avoid Prop Drilling

ComponentTree.jsx

```
import { useContext } from "react";
import { ColorContext } from "../../Context/MyContext";

function Parent() {
  return <Child />;
}

function Child() {
  return <GrandChild />;
}

function GrandChild() {
  const color = useContext(ColorContext);
  return <p style={{ color }}>I am {color}!</p>;
}

export default Parent;
```

More use cases for useContext

Pass User Info

```
const UserContext = React.createContext();

function App() {
  const user = { name: "Siddharth", role: "Admin" };

  return (
    <UserContext.Provider value={user}>
      <Navbar />
      <Dashboard />
    </UserContext.Provider>
  );
}

function Navbar() {
  const user = React.useContext(UserContext);
  return <p>Welcome, {user.name}!</p>;
}

function Dashboard() {
  const user = React.useContext(UserContext);
  return <p>Role: {user.role}</p>;
}
```

More use cases for useContext

Language
Localization

/

```
const LanguageContext = React.createContext();

function App() {
  const [lang, setLang] = React.useState("en");
  const translations = {
    en: { greeting: "Hello" },
    fr: { greeting: "Bonjour" }
  };

  return (
    <LanguageContext.Provider value={translations[lang]}>
      <Greeting />
      <button onClick={() => setLang(lang === "en" ? "fr" : "en")}>
        Switch Language
      </button>
    </LanguageContext.Provider>
  );
}

function Greeting() {
  const text = React.useContext(LanguageContext);
  return <h1>{text.greeting}</h1>;
}
```

More use cases for useContext

Theme Switching

```
const ThemeContext = React.createContext();

function App() {
  const [theme, setTheme] = React.useState("light");

  return (
    <ThemeContext.Provider value={theme}>
      <Header />
      <Content />
      <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
        Toggle Theme
      </button>
    </ThemeContext.Provider>
  );
}

function Header() {
  const theme = React.useContext(ThemeContext);
  return <h1 style={{ color: theme === "light" ? "black" : "white" }}>Header</h1>;
}
```

What is useReducer?

- A React hook that manages **state transitions** using a **reducer function**.
- Alternative to useState when:
 - State logic is **complex**.
 - Next state depends on **previous state**.
 - Multiple actions modify the same state.

Syntax : useReducer

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- **reducer**: A function `(state, action) => newState`.
- **initialState**: The starting state.
- **state**: Current state value.
- **dispatch(action)**: Function to trigger state updates.

Internal Flow of useReducer

1. Initialization

- React sets state = initialState.
- dispatch function is prepared.

1. Dispatching an Action

- You call dispatch with an action object.
- `dispatch({ type: "increment" });`

1. Reducer Execution

- React internally calls:
- `newState = reducer(currentState, action);`

1. State Update

- If `newState !== currentState`, React schedules a re-render.
- On re-render, state now holds `newState`.

1. Component Re-Renders

- UI updates with the new state.

Example : Counter

```
import { useReducer } from "react";

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return { count: 0 };
    default:
      return state;
  }
}
```


Example : Counter

```
function Counter() {  
  const [state, dispatch] = useReducer(reducer, initialState);  
  
  return (  
    <div>  
      <h1>Count: {state.count}</h1>  
      <button onClick={() => dispatch({ type: "increment" })}>+</button>  
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>  
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>  
    </div>  
  );  
}  
  
export default Counter;
```

Example : Form

```
import { useReducer } from "react";

const initialState = { name: "", email: "" };

function reducer(state, action) {
  switch (action.type) {
    case "setName":
      return { ...state, name: action.value };
    case "setEmail":
      return { ...state, email: action.value };
    case "reset":
      return initialState;
    default:
      return state;
  }
}
```

Example : Form

```
function Form() {  
  const [state, dispatch] = useReducer(reducer, initialState);  
  
  return (  
    <div>  
      <input  
        placeholder="Name"  
        value={state.name}  
        onChange={(e) => dispatch({ type: "setName", value: e.target.value })}  
      />  
      <input  
        placeholder="Email"  
        value={state.email}  
        onChange={(e) => dispatch({ type: "setEmail", value: e.target.value })}  
      />  
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>  
      <p>  
        Name: {state.name}, Email: {state.email}  
      </p>  
    </div>  
  );  
}  
  
export default Form;
```

Example : Todo App

```
import { useReducer, useState } from "react";

const initialState = [];

function reducer(state, action) {
  switch (action.type) {
    case "add":
      return [...state, { id: Date.now(), text: action.text, done: false }];
    case "toggle":
      return state.map((todo) =>
        todo.id === action.id ? { ...todo, done: !todo.done } : todo
      );
    case "remove":
      return state.filter((todo) => todo.id !== action.id);
    default:
      return state;
  }
}
```

Example : Todo App

```
function TodoApp() {
  const [todos, dispatch] = useReducer(reducer, initialState);
  const [text, setText] = useState("");

  return (
    <div>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Add todo"
      />
      <button
        onClick={() => {
          dispatch({ type: "add", text });
          setText("");
        }}
      >
        Add
      </button>

      <ul>
        {todos.map((todo) => (
          <li key={todo.id} style={{ textDecoration: todo.done ? "line-through" : "none" }} >
            {todo.text}
            <button onClick={() => dispatch({ type: "toggle", id: todo.id })}>Toggle</button>
            <button onClick={() => dispatch({ type: "remove", id: todo.id })}> X </button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default TodoApp;
```

Memo

- `React.memo` is a higher-order component (HOC).
- It wraps a functional component and tells React: “Only re-render this component if its props have changed.”
- Syntax:

```
const MemoizedComponent = React.memo(MyComponent);
```

Example: Without Memo

```
import { useState } from "react";

const Child = ({ value }) => {
  console.log("Child rendered");
  return <p>Value: {value}</p>;
};

function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Parent Count: {count}</h1>
      <button
        onClick={() => {
          setCount(count + 1);
        }}
      >
        Increment Parent
      </button>
      <Child value="Hello" /> {/* always re-renders */}
    </div>
  );
}

export default Parent;
```

Example: With Memo

```
import { useState } from "react";
import { memo } from "react";

// Child component wrapped with React.memo
const Child = memo(({ value }) => {
  console.log("Child rendered");
  return <p>Value: {value}</p>;
});

function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Parent Count: {count}</h1>
      <button
        onClick={() => {
          setCount(count + 1);
        }}
      >
        Increment Parent
      </button>
      { /* <Child> re-renders only if props change */ }
      <Child value="Hello" />
    </div>
  );
}

export default Parent;
```


useCallback and useMemo

- In React, components **re-render** whenever **state** or **props** change.
- This can cause:
 - Unnecessary recalculations (**useMemo** helps here).
 - Unnecessary re-creations of functions (**useCallback** helps here).

useCallback

- Returns a memoized version of a callback function.
- Prevents a new function from being created on every render.
- Function is recreated only if dependencies change.
- Helps avoid unnecessary re-renders of child components.
- Useful for performance optimization when passing callbacks as props.
- **Syntax:**

```
const memoizedFn = useCallback(fn, [dependencies]);
```

 - Returns a memoized version of a function.
 - Function reference remains the same until dependencies change.

Example: Without useCallback

```
import { useState, memo } from "react";

const Child = memo(({ onClick }) => {
  console.log("Child rendered");
  return <button onClick={onClick}>Click Me</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // without useCallback → new function created every render → child re-renders
  const handleClick = () => {
    console.log("Button clicked");
  };

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <Child onClick={handleClick} />
    </div>
  );
}

export default Parent;
```

Example: With useCallback

```
import { useState, memo, useCallback } from "react";

const Child = memo(({ onClick }) => {
  console.log("Child rendered");
  return <button onClick={onClick}>Click Me</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // with useCallback → new function created only if dependencies change
  const handleClick = useCallback(() => {
    console.log("Button clicked");
  }, []);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <Child onClick={handleClick} />
    </div>
  );
}

export default Parent;
```

useMemo

- Returns a memoized value (cached result of a calculation).
- Recomputes the value only if dependencies change.
- Prevents running expensive calculations on every render.
- Helps optimize performance when dealing with derived or computed data.
- Unlike `useCallback` (which memoizes functions), `useMemo` memoizes values/results.
- **Syntax:**

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

 - Memoizes the result of a calculation.
 - Only recomputes if dependencies change.

Example: Without useMemo

```
import { useState } from "react";
function Calculator({ num }) {
  const expensiveValue = () => {
    console.log("Calculating...");
    let result = 0;
    for (let i = 0; i < 1000000000; i++) {
      result += i % num;
    }
    return result;
  };
  return <p>Calculated Value: {expensiveValue()}</p>;
}

function ExpensiveCalculator() {
  const [num, setNum] = useState(5);
  const [count, setCount] = useState(0);

  return (
    <div>
      <Calculator num={num} />
      <p>Count: {count}</p>
      <button onClick={() => setNum(num + 1)}>Change Num</button>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
    </div>
  );
}

export default ExpensiveCalculator;
```

Example: With useMemo

```
import { useState } from "react";
import { useMemo } from "react";

function Calculator({ num }) {
  const expensiveValue = useMemo(() => {
    console.log("Calculating...");
    let result = 0;
    for (let i = 0; i < 1000000000; i++) {
      result += i % num;
    }
    return result;
  }, [num]); // only re-run if num changes
  return <p>Calculated Value: {expensiveValue}</p>;
}

function ExpensiveCalculator() {
  const [num, setNum] = useState(5);
  const [count, setCount] = useState(0);

  return (
    <div>
      <Calculator num={num} />
      <p>Count: {count}</p>
      <button onClick={() => setNum(num + 1)}>Change Num</button>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
    </div>
  );
}

export default ExpensiveCalculator;
```